Meisterware Detektor Viability Assessment

Executive summary

Creating Meisterware Detektor (a CI security scanner for AI agents) remains viable and would be meaningfully helpful to the AI agent community, but the product's success depends on deliberate differentiation and the avoidance of a "first scanner" positioning. Your uploaded baseline research correctly flags that agent security is becoming a "shift-left" CI problem and proposes an MVP centered on scanning tool-connected agents (especially MCP + skills) plus emitting a portable "security report" artifact. That direction aligns tightly with current, well-documented risk taxonomies and real-world incidents. [1]

The strongest evidence that Detektor would be *useful now* is that agentic risks are (a) acknowledged as core security risks by authoritative guidance, and (b) actively exploited where agents run with privileges—especially in CI and "agent extension" ecosystems (MCP servers, skills). [1]

In particular:

- **Prompt injection and agent abuse are treated as systemic failures, not prompt-tuning problems.** The National Cyber Security Centre argues LLMs do not enforce an internal boundary between "data" and "instructions," making prompt injection a residual risk that likely cannot be "fully fixed" like SQL injection; instead, systems should be designed to reduce impact via deterministic safeguards and privilege control. [2]
- **Agent supply chain risk is already severe in the wild.** Snyk reports scanning 3,984 agent skills and finding 13.4% with at least one critical issue; expanding to any severity, 36.82% had at least one security flaw, and they confirmed dozens of malicious payloads. [3]
- **CI + agent automation is demonstrably exploitable.** Aikido Security describes "PromptPwnd," where untrusted issue/PR content is embedded into prompts inside CI workflows, leading to privileged actions and potential secret leakage when an agent has access to high-privilege tokens/tools. [4]

At the same time, the competitive landscape has matured quickly: multiple tools (open-source and commercial) already scan key agent surfaces (MCP servers, skills, agent configs, and LLM-app code changes) and integrate into CI via exit-code gating and/or SARIF-based code scanning dashboards. [5]

Bottom line: Detektor is viable if positioned as a **unifying CI standard and cross-surface scanner** (configs + MCP + skills + CI-agent workflow hazards) with **portable**

artifacts, explicit threat-model mapping, and CI-grade outputs (SARIF + stable JSON + deterministic exit codes). The baseline's "spec-first artifact contract + scenario format + severity gating semantics + provenance" direction is the most defensible and strategically differentiating path.

Market need and demand

The core pain points are now CI-native and agent-specific

Modern "agentic" systems use tools, maintain memory/state, and perform actions—so security failures are not limited to bad text outputs; they include **unwanted side effects** like data exfiltration, permission misuse, and destructive actions. [1] OWASP's AI Agent Security Cheat Sheet explicitly frames agents as systems that reason/plan/use tools and highlights risks like prompt injection (direct/indirect), tool abuse/privilege escalation, data exfiltration, memory poisoning, excessive autonomy, cascading failures, and denial of wallet.[1]

These risks map directly to CI pain points because many of the "agent surfaces" are now **code-reviewed artifacts** and **supply chain inputs**:

- **Agent configs as code:** Files such as system instructions and tool permission configs define the effective attack surface. Mend.io explicitly argues that agent configurations ("agents as code") are becoming part of the SDLC and should be scanned like IaC, because misconfigurations can enable code execution, data exfiltration, and policy bypass. [6]
- **Agent extensions as code:** Skills and MCP servers are effectively "plugin ecosystems" with high privilege. MCP implementations often fetch tool descriptions and treat them as guidance, while skills and scripts can instruct agents to run commands, access files, or handle credentials. [7]
- **Agents inside CI:** Teams increasingly run AI agents to triage issues, label PRs, generate patches, or review code. When untrusted PR/issue content is inserted into prompts and the agent has high-privilege tooling, CI becomes an attack surface (PromptPwnd). [4]

Real-world incidents and research support "shift-left" agent security

The case for CI-first agent security is supported by both incident reports and peer-reviewed research:

- **Prompt injection as a residual, architectural risk:** The NCSC's 2025 analysis emphasizes that LLMs do not distinguish "instructions" vs "data" internally and

encourages risk reduction via privilege constraints and deterministic controls, especially for tool-calling systems. [2]

- **Adaptive attacks defeating defences:** A NAACL 2025 paper evaluates multiple indirect prompt injection defences and reports bypassing all eight with adaptive attacks, achieving >50% success rates, implying that defences must be evaluated continuously under evolving attacks. [8]
- **CI exploit path with prompt injection + privileged tools:** Aikido's PromptPwnd describes a repeatable failure mode: attacker-controlled issue/PR text → injected into model prompt → agent executes privileged tools in CI → possible secret leakage or workflow manipulation. [4]
- **Supply chain compromise around agent tooling:** The Cline incident demonstrates that AI-adjacent tooling and workflows can become supply chain targets; official post-mortem and advisory documentation describe a compromised publish token leading to an unauthorized version push and remediation steps emphasizing stronger provenance and publishing controls. [9]

Why the demand is not just "security theatre"

Demand signals are particularly strong because agent adoption is rapidly expanding and migrating into "ordinary engineering," enlarging the target surface:

- GitHub reports that more than **1.1 million public repositories** now use an LLM SDK (and 693,867 were created in the prior 12 months), indicating broad-based adoption of LLM/agent integration patterns that will need guardrails and scanning over time. [10]
- MCP's own security guidance explicitly indicates that **tool descriptions/annotations should be considered untrusted unless obtained from trusted servers**, and it emphasizes consent, privacy, and tool safety principles that organizations will need to operationalize in tooling. [7]

Taken together, these sources support a clear market need: teams need CI-grade tooling that detects and gates unsafe agent configurations and extensions before they land in production, and they need outputs compatible with existing DevSecOps workflows (PR annotations, SARIF dashboards, policy gates). [6]

Competitive landscape

Competitive segmentation: scanners exist, but coverage is fragmented

Your baseline research correctly observed that "a first scanner" claim is not supportable under broad definitions. The market is already populated by tools that scan specific surfaces (PR diffs, MCP servers, skills, configs) and integrate into CI/CD. [11]

The opportunity is to unify and standardize across surfaces with stable, portable artifacts and consistent policy semantics—i.e., to become “the CodeQL/SARIF-like contract” for agent security findings across MCP + skills + configs + CI-agent workflows, rather than “yet another single-surface scanner.” [12]

Competitor comparison table

Product / project	Primary surface(s)	CI integration style	Outputs	Notable strengths	Evidence
"Promptfoo" Code Scanning	PR/code changes for LLM app risks	GitHub App + GitHub Action; configurable minimum severity; PR review comments	PR review findings (platform-native); (docs emphasize workflow integration; SARIF not emphasized in the docs)	Agentic tracing of flows beyond diff; focuses on prompt injection, PII exposure, excessive agency, insecure output handling	[11]
Invariant Labs MCP-Scan	MCP server/tool metadata (tool descriptions, shadowing, rug pulls)	CLI; “run with a simple command”; registry integration partnerships	CLI report; includes tool pinning/hashing; involves local + API analysis	Strong MCP threat focus (tool poisoning, rug pulls, cross-origin escalation); explicit privacy disclosure about sharing tool descriptions with vendor for scans	[5]
Cisco Skill Scanner	Agent Skills packages (skills spec formats)	CLI; explicitly “CI/CD Ready”; exit codes for build failure; SARIF for GitHub code scanning	SARIF, JSON/markdown/table (per PyPI); exit-code gating	Multi-engine: pattern + behavioural dataflow + LLM analyser; explicit SARIF + policy handling	[13]
Cisco MCP Scanner	MCP servers/tools/prompts/resources;	CLI or REST API; includes offline scanning for CI and	JSON/table/summary etc (repo)	Multi-engine (YARA + LLM-as-judge + vendor inspect API); supports	[14]

	optional offline scanning per package description	air-gapped contexts (per PyPI)		scanning known configs across MCP clients	
Snyk ToxicSkills + mcp-scan	Agent Skills + MCP server scanning (and broader “agent supply chain”)	CLI usage promoted; enterprise rollout options referenced	CLI outputs (format not fully enumerated in captured view); integrates with broader platform	Large-scale ecosystem measurement; explicit taxonomy of skill threats; strong narrative for enterprise rollout	[3]
Mend.io Agent Configuration Scanning	“Agents as code” configs (CLAUDE.md/.cursorrules etc)	Integrated into Mend AI Scanner; “CI-friendly enforcement” positioning	Platform-integrated findings (grid + remediation guidance)	Treats configs as a control plane; explicit supported ecosystems list (Cursor, Claude Code, Copilot, Windsurf, OpenClaw)	[6]
AgentShield (Rust crate)	Agent extensions (MCP servers, OpenClaw skills, etc.)	GitHub Action recommended; fail-on threshold; SARIF upload	SARIF + JSON/HTML/console (per docs)	Offline-first; unified IR + detector rules; strong CI ergonomics via SARIF and Action	[15]

Feature matrix

Legend: ✓ = supported/explicitly documented; ~ = partial/indirect; blank = not indicated in primary docs above.

Capability	Promptfoo Code Scan	Invariant MCP-Scan	Cisco Skill Scanner	Cisco MCP Scanner	Snyk mcp-scan / ToxicSkills	Mend Agent Config Scan	AgentShield (Rust)
PR diff / code flow analysis	✓ [16]						
Agent config scanning (rules files/configs)	~ [16]			~ [17]	~ [18]	✓ [6]	~ [15]
MCP server scanning	~ [16]	✓ [5]		✓ [14]	✓ [18]		✓ [15]
Agent Skills scanning (SKILL.md ecosystem)			✓ [13]		✓ [3]	~ [6]	✓ [15]
Scenario-based adversarial testing							
Deterministic policy gates via exit codes	~ [11]	~ [5]	✓ [13]	~ [14]	~ [18]	~ [6]	✓ [15]
SARIF output for code scanning dashboards			✓ [13]				✓ [15]
Offline-first / no upload by default			~ [13]	~ [17]			✓ [15]
Artifact signing / provenance model							

Implication for Detektor: the matrix shows real coverage, but also a clear gap: very few tools unify across **(1) configs, (2) MCP, (3) skills, (4) CI-agent workflow hazards, (5) portable artifacts + provenance**, and essentially none standardize *scenario-based security regression tests* as portable CI assets.

Technical feasibility

Architecture feasibility and integration points

A CI security scanner for agents is technically feasible as a **hybrid static + controlled-dynamic analyser** that produces CI-native artifacts:

- CI systems already support treating scanner output as a hard gate via **exit codes**, and GitHub/GitLab/Azure Pipelines clearly document non-zero exit behaviour as a failure mechanism. [19]
- CI-native dashboards accept standardized formats like **SARIF 2.1.0** for code scanning alerts, enabling first-class developer ergonomics (inline annotations, PR alerts, security dashboards). [12]

A practical Detektor architecture can be organized around the agent surfaces implied by OWASP guidance and the MCP specification:

- **Configs and policy files**: detect dangerous patterns (auto-approval, wildcard permissions, unsafe command/curl|bash instructions, secrets handling, exfil endpoints), consistent with what Mend describes as the "AI control plane" problem. [6]
- **MCP server/tool scanning**: parse and analyse MCP tool definitions and annotations; MCP's own spec warns that tool descriptions/annotations should be treated as untrusted unless obtained from trusted servers. [7]
- **Agent Skills scanning**: parse SKILL.md and bundled scripts/resources; the Agent Skills specification describes the format as a portable folder structure centered on SKILL.md. [20]
- **CI-agent workflow scanning**: detect PromptPwnd-style patterns (untrusted PR/issue text injected into prompts, privileged tool access, unsafe execution paths) as described by Aikido. [4]

Proposed reference architecture diagram

Scalability, privacy, and threat modelling considerations

Scalability:
Most Detektor checks should be designed to be **fast and parallelizable**, because CI costs and lead time are adoption killers. Static analysis (configs/skills/MCP metadata) is naturally parallel across files and packages; only optional dynamic/scenario testing should invoke model calls and be budgeted. Denial-of-wallet is explicitly recognized as an agent risk, reinforcing the need for cost controls in any dynamic testing modes. [1]

Data privacy posture:
Privacy constraints are central because agent configs and skills often include sensitive operational information. MCP's spec emphasizes consent and data privacy, and agent scanning vendors sometimes transmit tool descriptions/metadata to APIs for analysis (Invariant explicitly discloses this behaviour). [7]
A viable community-friendly posture is:

- Default offline scanning for code/config/skills; optional "bring-your-own-model" semantic analysis; optional cloud only with explicit redaction policies.
- Explicit "what leaves the build" documentation and a hard "no secrets in logs" design, consistent with existing best practices and the reality of agent supply chain compromise. [3]

Threat model coverage (minimum expected set):
Detektor's threat model should map to vetted taxonomies:

- OWASP agent risks (prompt injection, tool abuse, data exfiltration, memory poisoning, excessive autonomy, cascading failures). [1]
- NCSC "confusable deputy" framing: focus on reducing impact through privilege boundaries and deterministic safeguards. [2]
- CI exploit patterns (PromptPwnd) and agent skill supply chain patterns (ToxicSkills). [4]

Product features and roadmap

MVP feature set

Your uploaded baseline proposes an MVP that scans MCP servers and tool-using agents, runs prompt-injection attack suites, and emits a signed "Agent Security Report." That remains a strong MVP shape, but it needs to be updated to reflect the current market: **Mend is scanning configs, Cisco/Snyk are scanning skills, and multiple MCP scanners exist**. [6]

A pragmatic MVP that is both differentiating and shippable:

- **Multi-surface static scan (P0):**
  - Agent config scanning for risky patterns in rules/config files (e.g., prompt override patterns, command execution patterns, exfil endpoints, permission escalation), similar in category to Mend but focused on CI-first open artifacts. [6]
  - MCP scanning of tool definitions/annotations with rules aligned to MCP's "untrusted tool annotations" guidance; basic detection for tool poisoning / shadowing patterns. [21]
  - Skills scanning compatible with the Agent Skills spec (SKILL.md + scripts), with secret detection and suspicious download patterns informed by ToxicSkills findings. [20]
  - CI-agent workflow scanning for PromptPwnd-style hazards (untrusted issue/PR content inserted into prompts + privileged tool access). [4]
- **CI-grade outputs (P0):**
  - SARIF 2.1.0 output for GitHub code scanning dashboards + structured JSON for portability. [12]
  - Deterministic "fail-on" semantics (exit code policy) as the primary gating mechanism. [19]
- **Spec-first artifact contract (P0→P1):**
  - A versioned report schema and stable finding IDs mapped to OWASP Agentic Top 10 categories (or a compatible mapping layer). [22]
- **Optional semantic analysis (P1):**
  - A pluggable LLM-as-judge mode (local or customer-hosted) for harder-to-catch prompt manipulation patterns—carefully framed as best-effort, not perfect, reflecting NCSC guidance that "filters" are not a full solution. [2]

Prioritized feature backlog

Priority	Feature	What it does	Why it matters	Dependencies / notes
P0	SARIF output + GitHub Action wrapper	Produce GitHub-native alerts and PR annotations	Matches how developers consume security findings; reduces adoption friction. [12]	SARIF 2.1.0 compliance required. [12]
P0	Deterministic policy gating	--fail-on severity threshold → exit codes	CI enforcement is the product's core value. [19]	Provide GitLab examples too. [23]
P0	Agent config scanner	Scan rules/config files for exfil, auto-approve, unsafe commands	Configs are now treated as a "control plane" and are actively marketed to be scanned. [6]	Start with common ecosystems cited by Mend. [6]
P0	Skills scanner (SKILL.md + scripts)	Detect prompt injection patterns, secrets, suspicious downloads, unsafe scripts	ToxicSkills shows high prevalence of serious issues in skills ecosystems. [3]	Align with Agent Skills spec. [20]
P0	MCP surface scanner	Scan MCP tool metadata and instructions; detect suspicious/unsafe tool descriptions	MCP spec warns tool annotations/behavior descriptions are untrusted unless trusted server. [21]	Must support popular MCP config locations over time
P0	CI-agent workflow scanner	Detect PromptPwnd hazards in CI workflows	Demonstrated exploit class in GitHub Actions/GitLab CI. [4]	Initial ruleset can be deterministic SAST-style
P1	Spec-first report schema + taxonomy	Publish versioned JSON schema + finding IDs + mapping to OWASP agentic risks	Differentiates from single-vendor scanners; enables ecosystem interoperability. [22]	Keep it narrow and testable; treat as "contract"
P1	Cross-surface correlation	Correlate findings across configs + skills + MCP to produce unified risk posture	Makes scan artifacts comparable and actionable across agent surfaces	Needs stable IDs + normalized evidence model
P1	Provenance signing	Sign scan reports; verify in CI	Aligns with the ecosystem trend toward attestations/hashes and provenance	Start with simple signature + build metadata
P1	Scenario-based security regression tests	Portable "agent security tests" (assertions about tool calls/exfil/policy bypass)	Bridges the gap between static scans and real behavior; responds to adaptive attack dynamics. [8]	Must be cost-bounded (denial-of-wallet risk). [1]

Pricing models consistent with “no specific constraint”

A plausible pricing strategy (aligned with open-community benefit and sustainable revenue) is:

- Open-source CLI + rulepacks: free local scanning, deterministic checks, SARIF/JSON output.
- Hosted “Detektor Cloud” tiers:
    - **Team**: policy management, dashboards, private rulepacks, CI integrations.
    - **Enterprise**: on-prem/self-hosted runner, audit logs, SSO, compliance evidence export, internal registry integration.
- Usage levers: per-repo (CI seats), per-scan run, or per-organization; dynamic/scenario testing priced by compute/model budget.

This aligns with the reality that CI scanning adoption is easiest when the base CLI is frictionless and value is in governance, policy, and portfolio-level reporting—mirroring how many AppSec tools monetize.

Go-to-market and business viability

Target users and adoption channels

A high-probability adoption wedge is **developer-facing CI integration** and “security badges for agent surfaces,” because developers already understand CI gates and code scanning dashboards.

Primary personas to target:

- Agent builders shipping tool-using agents (startups + internal platform teams).
- Security and DevSecOps teams trying to govern agent adoption.
- OSS maintainers distributing agents, skills, and MCP servers who need “trust signals.”

Channels and distribution:

- GitHub-first distribution (Action + SARIF) because it maps to existing security workflows. [12]
- Partnerships with marketplaces/registries (MCP registries are being built and are explicitly framed as addressing security risks from ecosystem fragmentation). [24]
- Content-led GTM anchored in real exploit classes (PromptPwnd, ToxicSkills, “configs as control plane”), with reproducible sample repos and a public ruleset.

Partnership angles that can accelerate credibility:

- Collaborate with agent ecosystem standard efforts (MCP spec implementers; registry maintainers). [7]
- Align evaluation language with OWASP's vendor evaluation criteria for red teaming/tooling to avoid being perceived as "jailbreak-only." [25]

TAM, SAM, SOM (with explicit assumptions)

Because "agent CI security scanning" is a new subcategory, sizing must be presented as ranges with clear assumptions.

TAM (adjacent market anchor):
Grand View Research estimates the global application security market at **$10.65B in 2025**. Detektor's category is a subset of AppSec (DevSecOps scanning) rather than the whole market, but this provides a credible ceiling. [26]

SAM (teams building LLM/agent software likely to need CI scanning):
GitHub reports **1.1M+ public repos** importing an LLM SDK (with strong YoY growth). Public repos are only a fraction of real enterprise software, so this is best interpreted as a "visible lower bound" on active LLM/agent engineering practice. [10]
A conservative SAM framing:

- If even **5–15%** of orgs building LLM/agent software adopt a dedicated agent-security CI scanner (plausible given OWASP/NCSC risk framing and visible ecosystem compromise), and average spend is **$1k–$20k/year** depending on team size and deployment, SAM plausibly spans **$100M–$1B+** in the medium term (range intentionally wide due to early category formation). [1]

SOM (12–18-month attainable slice):
With no budget constraints specified, a realistic early SOM is driven by distribution and credibility, not category size. If Detektor ships an OSS CLI + GitHub Action + SARIF and wins "default scanner" status in a few ecosystems (skills/MCP/configs), a plausible 12-month SOM is:

- **OSS/community:** 1,000–10,000 repos adopting the Action (free tier).
- **Paying customers:** ~50–200 teams (Team tier) and ~5–20 Enterprise pilots (on-prem / private policy packs), assuming strong product quality and partnerships.

These are targets, not forecasts; the key is that adoption scales with frictionless CI integration and trust signals.

Cost drivers and unit economics

Key cost drivers likely to dominate:

- Engineering for multi-ecosystem parsers and maintenance (configs/skills/MCP evolving quickly). [7]
- False-positive control and rule quality (alert fatigue is a known adoption killer in security). [3]
- If you offer dynamic/scenario-based testing with model calls, model/compute costs can dominate; denial-of-wallet is itself an agent risk, so the product must be cost-bounded by design. [1]

Risks, mitigations, and recommended next steps

Key risks and mitigations

Technical risk: false confidence (coverage gaps)

- Risk: Users may treat "passed scan" as "safe," while agent security remains a residual risk and adaptive attacks can bypass defences. [2]
- Mitigation: Make threat model coverage explicit; publish "what we do not cover"; incorporate continuous updates and regression tests; prioritize deterministic checks and privilege modelling over fragile string filters. [2]

Technical risk: ecosystem churn

- Risk: MCP spec revisions, new agent config formats, and evolving skills conventions can break scanners. [7]
- Mitigation: Plugin architecture for parsers/adapters (similar to how leading tools structure multi-format support); versioned rulesets; "best effort" detection for unknown formats that still flags high-risk patterns (exfil, curl|bash, base64). [13]

Market risk: crowded positioning

- Risk: Several vendors already cover individual surfaces (skills, MCP, configs, PR scanning). [3]
- Mitigation: Own the "unifying contract": spec-first schema + stable taxonomy + cross-surface correlation + CI-grade artifacts (SARIF/JSON) + provenance. This is the clearest differentiation not already strongly occupied by a single incumbent. [12]

Legal/regulatory risk: privacy and compliance expectations

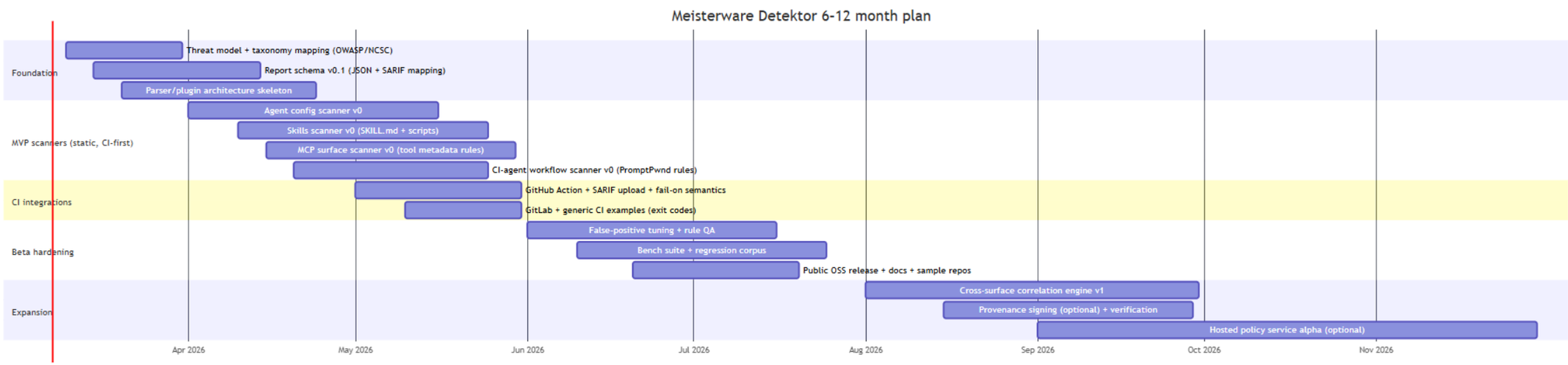
- Risk: Agent configs, skills, and tool metadata can include sensitive information. Moving data to a hosted scanner may create privacy and compliance exposure. [7]
- Mitigation: Default offline-first; explicit redaction; enterprise on-prem; align governance messaging with widely referenced frameworks. [27]

Regulatory context: the European Union AI Act is in force and is progressively applying through 2025–2027, with broad applicability of many rules by August 2026, which will increase demand for evidence, governance, and risk controls in relevant deployments. [28]

Recommended next steps with a 6–12-month timeline and resourcing

Your baseline suggests an MVP can be produced in roughly 6–10 weeks for a tight scope. That remains plausible if the MVP is sharply bounded to static multi-surface scanning + CI outputs (SARIF/JSON) and does not overcommit to expensive dynamic testing. [12]

6–12-month delivery timeline



Resource estimate (no budget constraint assumed)

A realistic staffing model to hit the above on schedule:

- 1 security-focused lead engineer (scanner core + policy engine)
- 1 engineer for parsers/integrations (skills/MCP/config ecosystems)
- 1 DevOps/CI engineer (Action packaging, SARIF, releases, test harness)
- 0.5 product/security research (taxonomy, rule design, threat intel updates, community)

This is roughly **3.5 FTE** for 6–9 months to reach a strong OSS + CI MVP and begin a hosted governance layer.

“Decision gates” to keep the product rigorous

To ensure the project stays viable and credibly helpful, adopt two hard internal gates:

- **Gate A (end of month 2):** SARIF + deterministic fail-on + at least two surfaces (configs + skills or configs + MCP) with low-noise findings in sample repos, demonstrated in GitHub Actions. [12]
- **Gate B (end of month 4):** Third surface + cross-surface correlation + published schema/taxonomy v0.1 + documented coverage limits + reproducible regression tests, aligned to OWASP agentic risks and NCSC “reduce impact” framing. [22]

Research Notes

Earlier versions of this research used the product name "Guardian".

The project was later renamed to "Detektor" to avoid naming conflicts and to align with the Meisterware product branding.

Citations

cheatsheetseries.owasp.org

1 AI Agent Security - OWASP Cheat Sheet Series https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html

ncsc.gov.uk

2 Prompt injection is not SQL injection (it may be worse) https://www.ncsc.gov.uk/pdfs/blog-post/prompt-injection-is-not-sql-injection.pdf

snyk.io

3 Snyk Finds Prompt Injection in 36%, 1467 Malicious Payloads in a ToxicSkills Study of Agent Skills Supply Chain Compromise | Snyk https://snyk.io/blog/toxicskills-malicious-ai-agent-skills-clawhub/

aikido.dev

4 Prompt Injection Inside GitHub Actions: The New Frontier of Supply Chain Attacks https://www.aikido.dev/blog/promptpwnd-github-actions-ai-agents

invariantlabs.ai

5 Introducing MCP-Scan: Protecting MCP with Invariant https://invariantlabs.ai/blog/introducing-mcp-scan

mend.io

6 Securing AI Agents: Static Scanning for AI Agent Configurations https://www.mend.io/blog/ai-agent-configuration-scanning/

modelcontextprotocol.io

7 Specification - Model Context Protocol https://modelcontextprotocol.io/specification/2025-11-25

21 Tools - Model Context Protocol https://modelcontextprotocol.io/specification/draft/server/tools

aclanthology.org

8 Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents - ACL Anthology https://aclanthology.org/2025.findings-naacl.395/

cline.bot

9 Post-mortem: Unauthorized Cline CLI npm publish on February 17, 2026 - Cline Blog https://cline.bot/blog/post-mortem-unauthorized-cline-cli-npm

github.blog

10 Octoverse: A new developer joins GitHub every second as AI leads TypeScript to #1 - The GitHub Blog https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/

promptfoo.dev

11 GitHub Action | Promptfoo https://www.promptfoo.dev/docs/code-scanning/github-action/

16 Code Scanning - LLM Security Vulnerability Scanner | Promptfoo https://www.promptfoo.dev/docs/code-scanning/

docs.github.com

12 About SARIF files for code scanning - GitHub Docs https://docs.github.com/en/code-security/concepts/code-scanning/sarif-files

19 Setting exit codes for actions - GitHub Docs https://docs.github.com/actions/creating-actions/setting-exit-codes-for-actions

pypi.org

13 cisco-ai-skill-scanner · PyPI https://pypi.org/project/cisco-ai-skill-scanner/

17 cisco-ai-mcp-scanner · PyPI https://pypi.org/project/cisco-ai-mcp-scanner/

18 mcp-scan · PyPI https://pypi.org/pypi/mcp-scan

github.com

14 GitHub - cisco-ai-defense/mcp-scanner: Scan MCP servers for potential threats & security findings. https://github.com/cisco-ai-defense/mcp-scanner

24 GitHub - modelcontextprotocol/registry: A community driven registry service for Model Context Protocol (MCP) servers. https://github.com/modelcontextprotocol/registry

docs.rs

15 agent-shield 0.2.0 - Docs.rs https://docs.rs/crate/agent-shield/0.2.0

agentskills.io

20 Specification - Agent Skills https://agentskills.io/specification

genai.owasp.org

22 OWASP Top 10 for Agentic Applications for 2026 - OWASP Gen AI Security Project https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

25 OWASP Vendor Evaluation Criteria for AI Red Teaming Providers & Tooling v1.0 - OWASP Gen AI Security Project https://genai.owasp.org/resource/owasp-vendor-evaluation-criteria-for-ai-red-teaming-providers-tooling-v1-0/

docs.gitlab.com

23 CI/CD YAML syntax reference | GitLab Docs https://docs.gitlab.com/ee/ci/yaml/

grandviewresearch.com

26 Application Security Market Size | Industry Report, 2033 https://www.grandviewresearch.com/industry-analysis/application-security-market

nist.gov

27 Artificial Intelligence Risk Management Framework (AI RMF 1.0) | NIST https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10

digital-strategy.ec.europa.eu

28 AI Act | Shaping Europe's digital future https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai